

Distributed Safe Navigation of Multi-Agent Systems Using Control Barrier Function-Based Controllers

Pol Mestres , Graduate Student Member, IEEE, Carlos Nieto-Granda , and Jorge Cortés 

Abstract—This letter proposes a distributed controller synthesis framework for safe navigation of multi-agent systems. We leverage control barrier functions to formulate collision avoidance with obstacles and teammates as constraints on the control input for a state-dependent network optimization problem that encodes team formation and the navigation task. Our algorithmic solution is valid under general assumptions for nonlinear dynamics and state-dependent network optimization problems with convex constraints and strongly convex objectives. The resulting controller is distributed, satisfies the safety constraints at all times, and asymptotically converges to the solution of the state-dependent network optimization problem. We illustrate its performance in a team of differential-drive robots in a variety of complex environments, both in simulation and in hardware.

Index Terms—Collision avoidance, decentralized control, motion planning.

I. INTRODUCTION

SAFETY-CRITICAL control has received a lot of attention in the robotics and controls communities motivated by applications like autonomous driving, navigation of robotic swarms, and optimal power flow of energy grids, where safety constraints are ubiquitous. Control barrier functions (CBFs) are a computationally efficient tool for the synthesis of safe controllers. In scenarios involving multiple autonomous agents, safety constraints often couple the decisions of the different team members. The challenge we face then is how to extend the CBF framework to multi-agent settings in a distributed manner (i.e., with each agent designing its local controller using only information from neighboring agents) while still retaining its efficiency, safety, and optimality guarantees. This question serves as the main motivation for this letter.

Literature Review: This work draws on notions and techniques from the body of work pertaining to CBFs [1], [2], which are a powerful tool to render safe a given set. Formally, safe controllers can be synthesized [3] by incorporating CBF-based conditions as constraints in an optimization problem whose objective encodes desired specifications on the input. Such optimization problems

have state-dependent constraints and need to be studied in feedback loop with a plant. Often, these problems cannot be solved instantaneously and instead approximate controllers must be implemented. Here, we draw on ideas from the “dynamical systems approach to algorithms” that views optimization algorithms as continuous-time dynamical systems, cf. [4], [5], which is a useful perspective when implementing optimization-based controllers in feedback systems [6], [7], [8], [9], [10]. In the context of multi-agent systems, many applications require a distributed implementation of the individual agents’ controllers. For optimization-based controllers resulting from CBFs, this requires a distributed solution of the resulting optimization problem. The works [11], [12] tackle this problem for quadratic programs (QPs) by splitting a centralized QP into local QPs that can be solved efficiently while preserving safety guarantees. However, the solution of these local QPs might lack optimality guarantees with respect to the original QP. The recent works [13], [14], [15], [16], [17] introduce different algorithms to solve constrained optimization problems in a distributed fashion while satisfying the constraints throughout the execution of the algorithm. However, [13], [14], [15], [16] are restricted to a class of parametric QPs with conditions on the coefficients of the affine constraints. This approach has recently been successfully implemented for the problem of global connectivity maintenance [18]. On the other hand, our previous work [17] does not consider the implementation of the optimization problem in feedback loop with a plant.

Statement of Contributions: We design a distributed controller for safe navigation of multi-agent systems. We propose a synthesis framework which leverages CBFs to formulate obstacle avoidance and inter-agent collision avoidance constraints as affine inequalities in the control input. These constraints are included in a state-dependent network optimization problem that finds the control inputs allowing the agents to reach different waypoints of interest while maintaining a given formation and satisfying the safety constraints. Our first contribution leverages the particular structure of the optimization problem to decouple its constraints by using a set of auxiliary variables while still keeping the same feasible set. This enables us to derive a distributed update law for these auxiliary variables that allows each agent to obtain its local control input by solving a local optimization problem without the need to coordinate with any other agent. Our second contribution establishes that the proposed controller design is distributed, safe, and asymptotically converges to the solution of the state-dependent network optimization problem. Our last contribution is the implementation of the proposed controller in a variety of different environments, robots and formations, both in simulation and in real hardware.

Manuscript received 7 February 2024; accepted 31 May 2024. Date of publication 13 June 2024; date of current version 17 June 2024. This letter was recommended for publication by Associate Editor L. Liu and Editor M. Ani Hsieh upon evaluation of the reviewers’ comments. This work was supported by the Tactical Behaviors for Autonomous Maneuver (TBAM) under Grant ARL-W911NF-22-2-0231. (Corresponding author: Pol Mestres.)

Pol Mestres and Jorge Cortés are with the Contextual Robotics Institute and the Department of Mechanical and Aerospace Engineering, UC San Diego, La Jolla, CA 92037 USA (e-mail: pomestre@ucsd.edu; cortes@ucsd.edu).

Carlos Nieto-Granda is with the U.S. Army Combat Capabilities Development Command Army Research Laboratory (ARL), Adelphi, MD 20783 USA (e-mail: carlos.p.nieto2.civ@army.mil).

Digital Object Identifier 10.1109/LRA.2024.3414268

II. PRELIMINARIES

We introduce here basic notation and preliminaries on CBFs, constrained optimization, and projected saddle-point dynamics. This section can be safely skipped by a reader already familiar with these notions.

Notation: We denote by $\mathbb{R}$ and $\mathbb{R}_{\geq 0}$ the set of real and non-negative real numbers, respectively. For a positive integer n , we let $[n] = \{1, \dots, n\}$. Given a set A , $|A|$ denotes its cardinality. Given $x \in \mathbb{R}^n$, $\|x\|$ denotes its Euclidean norm. For $a \in \mathbb{R}$ and $b \in \mathbb{R}_{\geq 0}$, we let

$$[a]_b^+ = \begin{cases} a, & \text{if } b > 0, \\ \max\{0, a\}, & \text{if } b = 0. \end{cases}$$

Given $a \in \mathbb{R}^n$ and $b \in \mathbb{R}_{\geq 0}^n$, $[a]_b^+$ denotes the vector whose i -th component is $[a_i]_{b_i}^+$, for $i \in [n]$. Given a set $\mathcal{P} \subseteq \mathbb{R}^n$ and variables $\xi = \{x_{i_j}\}_{j=1}^k$, we denote by $\Pi_{\xi} \mathcal{P} = \{(x_{i_1}, x_{i_2}, \dots, x_{i_k}) \in \mathbb{R}^k : x \in \mathcal{P}\}$ the projection of $\mathcal{P}$ onto the ξ variables. An undirected graph is a pair $\mathcal{G} = (V, \mathcal{E})$, where $V = V(\mathcal{G}) = [N]$ is the vertex set and $\mathcal{E} = \mathcal{E}(\mathcal{G}) \subseteq V \times V$ is the edge set, with $(i, j) \in \mathcal{E}$ if and only if $(j, i) \in \mathcal{E}$. The set $\mathcal{N}_i = \{j \in V : (i, j) \in \mathcal{E}\}$ denotes the neighbors of node i . For a function $f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, we denote by $\nabla_x f$ and $\nabla_y f$ the column vectors of partial derivatives of f with respect to the first and second arguments, respectively. Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be locally Lipschitz. A set $\mathcal{C}$ is forward invariant for the dynamical system $\dot{x} = F(x)$ if all trajectories that start in $\mathcal{C}$ remain in $\mathcal{C}$ for all positive times. A function $\alpha : \mathbb{R} \rightarrow \mathbb{R}$ is of extended class $\mathcal{K}_{\infty}$ if $\alpha(0) = 0$, α is strictly increasing, and $\lim_{t \rightarrow \infty} \alpha(t) = \infty$.

Control Barrier Functions: Consider the control-affine system

$$\dot{x} = F(x) + G(x)u \quad (1)$$

where $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $G : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ are locally Lipschitz functions, with $x \in \mathbb{R}^n$ the state and $u \in \mathbb{R}^m$ the input. Let $\mathcal{C} \subset \mathbb{R}^n$ and $h : \mathbb{R}^n \rightarrow \mathbb{R}$ be a continuously differentiable function such that

$$\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\}, \quad \partial \mathcal{C} = \{x \in \mathbb{R}^n : h(x) = 0\}. \quad (2)$$

The function h is a control barrier function (CBF) [1, Definition 2] of $\mathcal{C}$ if there exists an extended class $\mathcal{K}_{\infty}$ function α and $\mathcal{D} \subset \mathbb{R}^n$ with $\mathcal{C} \subset \mathcal{D}$ such that for all $x \in \mathcal{D}$, there exists $u \in \mathbb{R}^m$ with

$$L_F h(x) + L_G h(x)u + \alpha(h(x)) \geq 0. \quad (3)$$

A Lipschitz controller $k : \mathbb{R}^n \rightarrow \mathbb{R}^m$ such that $u = k(x)$ satisfies (3) for all $x \in \mathcal{C}$ makes $\mathcal{C}$ forward invariant. Hence, CBFs provide a way to guarantee safety.

Projected Saddle-Point Dynamics: Given continuously differentiable functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}^p$, whose derivatives are locally Lipschitz, consider the constrained nonlinear program

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g(x) \leq 0. \end{aligned} \quad (4)$$

Let $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}_{\geq 0}^p \rightarrow \mathbb{R}$ be its associated Lagrangian, $\mathcal{L}(x, \lambda) = f(x) + \lambda^T g(x)$. The *projected saddle-point dynamics* for $\mathcal{L}$ are

defined as follows:

$$\dot{x} = -\nabla_x \mathcal{L}(x, \lambda, \mu), \quad (5a)$$

$$\dot{\lambda} = [\nabla_{\lambda} \mathcal{L}(x, \lambda, \mu)]_{\lambda}^+. \quad (5b)$$

If f is strongly convex and g is convex, then $\mathcal{L}$ has a unique saddle point, which corresponds to the KKT point of (4). Moreover, [19, Theorem 5.1] ensures that it is globally asymptotically stable under the dynamics (5).

Constraint Mismatch Variables: Given $N \in \mathbb{Z}_{>0}$, consider a network composed by agents $\{1, \dots, N\}$ whose communication topology is described by a connected undirected graph $\mathcal{G}$. An edge (i, j) represents the fact that agent i can receive information from agent j and vice versa. For each $i \in [N]$ and $k \in [p]$, where $p \in \mathbb{Z}_{>0}$, let $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ be a strongly convex function and $g_i^k : \mathbb{R}^n \rightarrow \mathbb{R}$ a convex function. We consider the following optimization problem with separable objective function and constraints

$$\begin{aligned} \min_{u \in \mathbb{R}^{nN}} \quad & \sum_{i=1}^N f_i(u_i) \\ \text{s.t.} \quad & \sum_{i \in V(\mathcal{G}_k)} g_i^k(u_i) \leq 0, \quad k \in [p], \end{aligned} \quad (6)$$

where $\mathcal{G}_k$ is a connected subgraph of $\mathcal{G}$ for each $k \in [p]$. For each $k \in [p]$, the constraint in (6) couples the local variable u_i of agent i with the variables of all the other agents in $\mathcal{G}_k$. The constraints in (6) can be decoupled by introducing *constraint-mismatch variables* [17, 20], which help agents keep track of local constraints while collectively satisfying the original constraints. Specifically, we add one constraint-mismatch variable per agent and constraint. We let $z_i^k \in \mathbb{R}$ be the constraint-mismatch variable for agent i and constraint k . Let $P_i := \{k \in [p] : i \in V(\mathcal{G}_k)\}$ be the indices of the constraints involving agent i . For convenience, we let $u = [u_1, \dots, u_N]$, $z_i = \{z_i^k\}_{k \in P_i}$, $z = [z_1, \dots, z_N]$ and $q := \sum_{i=1}^N |P_i|$. Next, consider the problem

$$\begin{aligned} \min_{u \in \mathbb{R}^{nN}, z \in \mathbb{R}^q} \quad & \sum_{i=1}^N f_i(u_i) \\ \text{s.t.} \quad & g_i^k(u_i) + \sum_{j \in \mathcal{N}_i \cap V(\mathcal{G}_k)} (z_i^k - z_j^k) \leq 0, \quad i \in V(\mathcal{G}_k), \quad k \in [p]. \end{aligned} \quad (7)$$

In (7), the constraints are now locally expressible, meaning that agent $i \in [N]$ can evaluate the ones in which its variable u_i is present by using variables obtained from its neighbors. The next result establishes the equivalence of (6) and (7).

Proposition 2.1: (Equivalence between the two formulations): Let $\mathcal{F}^*$ be the solution set of (7). Then, $u^* = \Pi_u(\mathcal{F}^*)$ is the optimizer of (6).

The proof is similar to [17, Proposition 4.1], with the necessary modifications to account for the fact that the constraints of (6) might only involve a subset of the agents.

III. PROBLEM STATEMENT

We are interested in designing distributed controllers that allow a team of differential-drive robots to safely navigate an environment while maintaining a desired formation and visiting

a sequence of waypoints of interest. The robots have identities $\{1, \dots, N\}$ and follow unicycle dynamics

$$\dot{x}_i = v_i \cos \theta_i, \quad \dot{y}_i = v_i \sin \theta_i, \quad \dot{\theta}_i = \omega_i, \quad (8)$$

where $s_i = [x_i, y_i] \in \mathbb{R}^2$ is the position of agent i , $\theta_i \in [0, 2\pi)$ its heading and $v_i \in \mathbb{R}$ and $\omega_i \in \mathbb{R}$ are its linear and angular velocity control inputs, respectively.

We next leverage CBFs to encode the different safety specifications, giving rise to affine constraints in the control inputs. We note that, under the dynamics (8), direct application of (3) on functions that only depend on position results in limited design flexibility because ω_i does not appear in the time derivative of s_i . Instead, we follow [21, Section IV] and define, for all $i \in [N]$,

$$R(\theta_i) = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \\ \sin \theta_i & \cos \theta_i \end{bmatrix}, \quad p_i = s_i + lR(\theta_i)e_1, \quad L = \begin{bmatrix} 1 & 0 \\ 0 & 1/l \end{bmatrix}$$

where $e_1 = [1, 0]^T$ and $l > 0$ is a design parameter. This defines p_i as a point orthogonal to the wheel axis of the robot. It follows that $\dot{p}_i = R(\theta_i)L^{-1}u_i$, where $u_i = [v_i, \omega_i]^T$. Hence, by choosing p_i as our position variable, both control inputs appear in the time derivative of the position, which allows for more flexibility in our control design.

Avoiding obstacles: We consider an environment with M obstacles $\{\mathcal{O}_k\}_{k \in [M]}$, each expressible as the sublevel set of a differentiable function h_k , i.e., $\mathcal{O}_k = \{(x, y) \in \mathbb{R}^2 : h_k(x, y) < 0\}$. To guarantee that the robot does not collide with the obstacles, we want to ensure that

$$h_k(p_i) \geq \eta_k > 0, \quad \forall k \in [M]. \quad (9)$$

The parameter $\eta_k \in \mathbb{R}$ should be taken large enough to guarantee that the whole physical robot (instead of just p_i) does not collide with the obstacle $\mathcal{O}_k$. For instance, if r_i is the radius of robot i and $\mathcal{O}_k$ is a circular obstacle with center at $m_k \in \mathbb{R}^2$ and radius R_k so that $h_k(p_i) = \|p_i - m_k\|^2 - R_k^2$, then η_k can be taken as $(r_i + l)^2 + 2R_k(r_i + l)$. The CBF condition associated with (9) with linear extended class $\mathcal{K}_\infty$ function with slope $\alpha_k > 0$ reads

$$\nabla h_k(p_i)^T R(\theta_i)L^{-1}u_i \geq -\alpha_k(h_k(p_i) - \eta_k). \quad (10)$$

Avoiding inter-agent collisions: We also want to enforce that agents do not collide with other team members. To achieve this, we assume it is enough for each robot i to avoid colliding with a subset of the agents $N_i \subset [N] \setminus \{i\}$. This is motivated by the fact that our control design will make the team maintain a formation at all times. Hence, agent only needs to avoid colliding with agents closest to it in the formation. For this reason, we assume that if $j \in N_i$, then $i \in N_j$, and agent i can communicate with all agents in N_i to obtain their state variables. We assume that the resulting communication graph is connected. For any $i \in [N]$ and $j \in N_i$, we want to ensure

$$d(p_i, p_j) := \|p_i - p_j\|^2 - d_{\min}^2 \geq 0, \quad (11)$$

with $d_{\min} \geq r_i + r_j + 2l$. The CBF condition for (11) with a linear extended class $\mathcal{K}_\infty$ function with slope $\alpha_c^{ij} > 0$ reads

$$2(p_i - p_j)^T (R(\theta_i)L^{-1}u_i - R(\theta_j)L^{-1}u_j) \geq -\alpha_c^{ij}d(p_i, p_j). \quad (12)$$

Team formation: Finally, we are interested in making the team reach a goal while maintaining a certain formation. To do so, we

define a *leader* for the team (without loss of generality, agent 1). The rest of the agents $\{2, \dots, N\}$ are referred to as *followers*. The leader is in charge of steering the team towards a given waypoint $q_1 \in \mathbb{R}^2$. To achieve it, we define a nominal controller $u_{\text{nom},1} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ that steers it towards q_1 . We use the stabilizing controller for the unicycle dynamics in [22]. To define it, let $e_1 : \mathbb{R}^2 \rightarrow \mathbb{R}$ and $\beta_1 : \mathbb{R}^2 \rightarrow \mathbb{R}$ defined as

$$e_1(p_1) = \|p_1 - q_1\|, \quad \beta_1(p_1) = \arctan \left(\frac{p_{1,y} - q_{1,y}}{p_{1,x} - q_{1,x}} \right).$$

The control law is $u_{\text{nom},1}(p_1) = [v_1(p_1), \omega_1(p_1)]$, where

$$v_1(p_1) = k_r e(p_1) \cos(\beta_1(p_1)) - \theta_1, \quad (13a)$$

$$\omega_1(p_1) = k_a \beta_1(p_1) + \frac{k_r}{2} \sin(2\beta_1(p_1)) \frac{\beta_1(p_1) + h\theta_1}{\beta_1(p_1)}, \quad (13b)$$

and $k_r > 0$, $k_a > 0$ and $h > 0$ are design parameters. We can also specify a sequence of waypoints for the leader: once the leader is within a given tolerance of the current waypoint, $u_{\text{nom},1}$ can be updated to steer it towards the next waypoint.

As the leader moves towards the desired waypoint, the followers follow it while maintaining a certain formation of interest. We define the desired formation positions for the followers as follows. First, recall that $N_1 := \{i \in [N] \setminus \{1\} : i \text{ and } 1 \text{ can communicate their respective state variables}\}$ and, for $k \in \mathbb{Z}_{>0}$, $k > 1$, define the k -neighborhood of the leader as $N_1^k := \{i \in [N] : \exists j \in N_1^{k-1} \text{ s.t. } i \in N_j\}$ (here, $N_1^1 = N_1$). Since the communication graph is connected, there exists $K \in \mathbb{Z}_{>0}$ such that, for all $i \in [N] \setminus \{1\}$, there is $k \in [K]$ such that $i \in N_1^k$. For every $i \in [N] \setminus \{1\}$, we let k_i be the smallest positive integer k such that $i \in N_1^k$. For every $i \in [N] \setminus \{1\}$, we consider functions $q_i : \mathbb{R}^{2|N_1^{k_i-1} \cap N_i|} \rightarrow \mathbb{R}^2$ such that $q_i(\{p_j\}_{j \in N_1^{k_i-1} \cap N_i})$ defines the desired formation position for agent i . Agent i aims to remain as close as possible to $q_i(\{p_j\}_{j \in N_1^{k_i-1} \cap N_i})$ while maintaining the safety constraints. To achieve this, we define a nominal controller $u_{\text{nom},i} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ for each agent $i \in \{2, \dots, N\}$ analogous to (13) that steers it towards $q_i(\{p_j\}_{j \in N_1^{k_i-1} \cap N_i})$. Note that q_i can be computed in a distributed fashion because it only depends on the positions of agents in $N_1^{k_i-1} \cap N_i \subseteq N_i$.

Agents collectively try to design controllers $\{u_i\}_{i=1}^N$ that satisfy the obstacle avoidance and inter-agent collision avoidance constraints while remaining as close as possible to their nominal controller. By employing weighting matrix-valued functions $\Gamma_i : \mathbb{R}^n \rightarrow \mathbb{R}^{m \times m}$ for each $i \in [N]$ that can be designed to penalize the linear and angular velocity inputs differently, and leveraging the CBF conditions (10), (12), we obtain the following network-wide optimization problem:

$$\begin{aligned} \min_{\{u_i \in \mathbb{R}^2\}_{i=1}^N} \quad & \sum_{i=1}^N \frac{1}{2} \|\Gamma_i(p_i)(u_i - u_{\text{nom},i}(p_i))\|^2 \\ \text{s.t.} \quad & (p_i - p_j)^T (R(\theta_i)L^{-1}u_i - R(\theta_j)L^{-1}u_j) \geq -\alpha_c^{ij}d(p_i, p_j), \\ & \nabla h_k(p_i)^T R(\theta_i)L^{-1}u_i \geq -\alpha_k(h_k(p_i) - \eta_k), \\ & i \in [N], j \in N_i, k \in [M]. \end{aligned} \quad (14)$$

The presence of the controls u_i and u_j in the inter-agent collision avoidance constraints (12) means that the satisfaction of such constraints requires coordination between agents involved in

the constraint. This, together with the state-dependency of the objective function and constraints, poses challenges in the implementation of a distributed algorithm that solves (14). Our goal is to design a distributed controller that satisfies the constraints at all times and with the same optimality properties as the solution obtained directly from (14).

IV. DISTRIBUTED CONTROLLER DESIGN

In this section we design a distributed algorithmic solution to the problem formulated in Section III. As it turns out, our solution is valid for a more general setup, as we explain next. Assume that the agents' communication network is described by a connected undirected graph $\mathcal{G}$, as in Section III. An edge (i, j) represents the fact that agent i can receive information from agent j and vice versa. We describe the dynamics of each agent $i \in [N]$ by

$$\dot{\xi}_i = F_i(\xi_i, u_i) \quad (15)$$

where $F_i : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ is a locally Lipschitz function for each $i \in [N]$, $\xi_i \in \mathbb{R}^n$ is the state variable of agent i and $u_i \in \mathbb{R}^m$ is its local control input. Additionally, let $f_i : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ and $g_i^k : \mathbb{R}^{n|\mathcal{N}_i|} \times \mathbb{R}^m \rightarrow \mathbb{R}$, with $i \in [N]$ and $k \in [p]$ be functions satisfying the following.

Assumption 1: (Regularity and convexity of the optimization problem): For all $i \in [N]$ and $k \in [p]$, f_i and g_i^k are continuously differentiable functions with Lipschitz derivatives. We assume that for all $i \in [N]$ and $\xi_i \in \mathbb{R}^n$, the functions $f_i(\xi_i, \cdot)$ are strongly convex, and for all $i \in [N]$, $k \in [p]$ and $\xi_i \in \mathbb{R}^{n|\mathcal{N}_i|}$, the functions $g_i^k(\xi_i, \cdot)$ are convex.

The agents need to coordinate to solve an optimization problem of the form

$$\begin{aligned} \min_{\{u_i \in \mathbb{R}^m\}_{i=1}^N} \quad & \sum_{i=1}^N f_i(\xi_i, u_i), \\ \text{s.t.} \quad & \sum_{i \in V(\mathcal{G}_k)} g_i^k(\xi_{\mathcal{N}_i}, u_i) \leq 0, \quad k \in [p]. \end{aligned} \quad (16)$$

where $\xi_{\mathcal{N}_i} = \{\xi_j\}_{j \in \mathcal{N}_i}$ and $\mathcal{G}_k$ is a connected subgraph of $\mathcal{G}$ for each $k \in [p]$. Note that (14) is a particular case of (16), where $f_i(\xi_i, u_i) = \frac{1}{2} \|\Gamma_i(\xi_i)(u_i - u_{\text{nom},i}(\xi_i))\|^2$ and $\mathcal{G}_k$ corresponds to the graph with nodes $\{i\} \cup \{j\}$ and an edge between $\{i\}$ and $\{j\}$ for the inter-agent collision avoidance constraint between agents i and j , and $\mathcal{G}_k$ corresponds to the singleton $\{i\}$ for the obstacle avoidance constraints of agent i . Moreover, for the inter-agent collision avoidance constraint between agents i and j ,

$$\begin{aligned} g_i^k(p_i, \theta_i, p_j, \theta_j, u_i) &= -2(p_i - p_j)^T R(\theta_i) L^{-1} u_i \\ &\quad - \alpha_c^{ij} d(p_i, p_j), \\ g_j^k(p_j, \theta_j, p_i, \theta_i, u_j) &= -2(p_j - p_i)^T R(\theta_j) L^{-1} u_j \\ &\quad - \alpha_c^{ij} d(p_i, p_j). \end{aligned}$$

For the collision avoidance constraint of agent i with the k th obstacle, $g_i^k(p_i, \theta_i, u_i) = -\nabla h_k(p_i)^T R(\theta_i) L^{-1} u_i - \alpha_k(h_k(p_i) - \eta_k)$. Problem (16) can encode other types of safety constraints and more general dynamics. We henceforth denote

$\xi = [\xi_1, \dots, \xi_N] \in \mathbb{R}^{nN}$ and $u = [u_1, \dots, u_N] \in \mathbb{R}^{mN}$. We assume that (16) is feasible.

Assumption 2: Problem (16) is feasible for all $\xi \in \mathbb{R}^{nN}$.

Assumption 2 is necessary for the solution of (16) to be well defined for all $\xi \in \mathbb{R}^{nN}$. If the constraints of (16) are defined by CBFs, such as those in (14), their joint feasibility can be characterized, cf. [23], [24], [25].

We let $u^* : \mathbb{R}^{nN} \rightarrow \mathbb{R}^{mN}$ be the function mapping each $\xi \in \mathbb{R}^{nN}$ to the solution of (16). To tackle the design of our distributed algorithm, we first deal with the coupling induced by the inequality constraints by introducing *constraint mismatch variables* z_i^k for each agent and constraint in which it is involved. We use the same notation as in Section II and reformulate (16) as

$$\begin{aligned} \min_{u \in \mathbb{R}^{mN}, z \in \mathbb{R}^q} \quad & \sum_{i=1}^N f_i(\xi_i, u_i), \\ \text{s.t.} \quad & g_i^k(\xi_{\mathcal{N}_i}, u_i) + \sum_{j \in \mathcal{N}_i \cap V(\mathcal{G}_k)} (z_i^k - z_j^k) \leq 0, \quad i \in V(\mathcal{G}_k), \quad k \in [p]. \end{aligned} \quad (17)$$

In order to facilitate the analysis of the convergence properties of the algorithms that will follow, we regularize (17) by adding the term $\epsilon \sum_{k=1}^p \sum_{j \in V(\mathcal{G}_k)} (z_j^k)^2$, with $\epsilon > 0$, in the objective function of (17). Details regarding this regularization are covered in the Appendix. With the added regularization term, the problem (17) has a strongly convex objective function and convex constraints, and therefore has a unique optimizer for every $\xi \in \mathbb{R}^{nN}$. Let $u^{*,\epsilon} : \mathbb{R}^{nN} \rightarrow \mathbb{R}^{mN}$ and $z^{*,\epsilon} : \mathbb{R}^{nN} \rightarrow \mathbb{R}^q$ be the functions mapping each $\xi \in \mathbb{R}^{nN}$ to the corresponding unique optimizers in u and z , respectively, of the regularized problem. We also let $\lambda^{*,\epsilon} : \mathbb{R}^{nN} \rightarrow \mathbb{R}^q$ be the function mapping each $\xi \in \mathbb{R}^{nN}$ to the optimal Lagrange multiplier of the regularized problem. Problem (17) (or its regularized version) cannot be decoupled into N local optimization problems because the optimal values of z in (17) require coordination among agents. However, for every fixed z , (17) can be solved locally by agent i by just optimizing over u_i as follows:

$$\begin{aligned} \min_{u_i \in \mathbb{R}^m} \quad & f_i(\xi_i, u_i), \\ \text{s.t.} \quad & g_i^k(\xi_{\mathcal{N}_i}, u_i) + \sum_{j \in \mathcal{N}_i \cap V(\mathcal{G}_k)} (z_i^k - z_j^k) \leq 0, \quad k \in [p]. \end{aligned} \quad (18)$$

We let $\bar{u}_i : \mathbb{R}^{n|\mathcal{N}_i|} \times \mathbb{R}^{|\mathcal{N}_i|} \rightarrow \mathbb{R}^m$ be the function that maps every $(\xi_{\mathcal{N}_i}, z_{\mathcal{N}_i}) \in \mathbb{R}^{n|\mathcal{N}_i|} \times \mathbb{R}^{\sum_{j \in \mathcal{N}_i} |P_j|}$ to the optimizer of (18), and $\bar{u} = [\bar{u}_1, \dots, \bar{u}_N]$. By construction, the controller $\bar{u}_i$ is **distributed**, as it only depends on variables which can be obtained through communication with agents in $\mathcal{N}_i$.

However, $\bar{u}_i$ depends on the chosen value of z . Since (17) contains a minimization over both u and z , $\bar{u}_i$ coincides with the optimizer over u of (16), one must also optimize over the *constraint mismatch variables* z . We do this by updating them with the projected saddle-point dynamics of the regularization of (17), cf. Fig. 1. We assume that the projected saddle-point dynamics can be run at a faster rate than the plant. Introducing a timescale separation parameter $\tau > 0$ to model this, the interconnection of the projected saddle-point dynamics with the

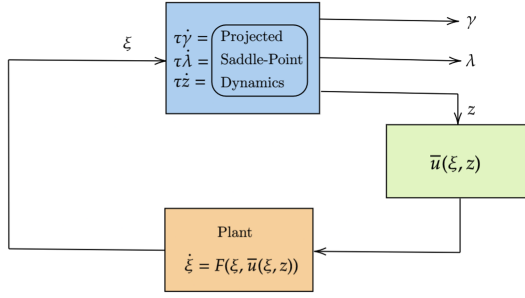


Fig. 1. Block diagram of (19). The blue block updates the variables γ , z and λ using the projected saddle-point dynamics of the regularized version of (17). In parallel, the plant is updated using the controller $\bar{u}$.

plant leads to the following dynamical system:

$$\tau \dot{\gamma}_i = -\nabla_{\gamma_i} f_i(\xi_i, \gamma_i) - \sum_{k \in P_i} \lambda_i^k \nabla_{\gamma_i} g_i^k(\xi_{N_i}, \gamma_i), \quad (19a)$$

$$\tau \dot{z}_i^k = -\epsilon z_i^k - \sum_{j \in N_i \cap \mathcal{G}_k} (\lambda_i^k - \lambda_j^k), \quad (19b)$$

$$\tau \dot{\lambda}_i^k = \left[g_i^k(\xi_{N_i}, \gamma_i) + \sum_{j \in N_i \cap \mathcal{G}_k} (z_j^k - z_i^k) \right]_{\lambda_i^k}^+, \quad (19c)$$

$$\dot{\xi}_i = F_i(\xi_i, \bar{u}_i(\xi_i, z_{N_i})), \quad (19d)$$

for all $i \in [N]$ and $k \in P_i$. We henceforth denote $\gamma = [\gamma_1, \dots, \gamma_N]$, $\lambda_i = \{\lambda_i^k\}_{k \in P_i}$ for all $i \in [N]$ and $\lambda = [\lambda_1, \dots, \lambda_N]$. The auxiliary variables γ and λ play the role of the primal variable u and the Lagrange multipliers of the constraints in (17), respectively. Our proposed algorithmic solution is the controller $\bar{u}_i$ for all $i \in [N]$ implemented for the current value of the state variables ξ_i and z_{N_i} in (19).

V. ANALYSIS OF THE SOLUTION: DISTRIBUTED CHARACTER, SAFETY, AND STABILITY

In this section we establish the properties of the controller proposed in Section IV. Throughout this section we use the same communication graph $\mathcal{G}$ introduced in Section IV. First we introduce various assumptions regarding problem (18) and discuss their sensibleness.

Assumption 3: (Feasibility of optimization problem): For all $i \in [N]$, the optimization problem (18) is feasible for all $\xi_{N_i} \in \mathbb{R}^{n|N_i|}$ and $z_{N_i} \in \mathbb{R}^{\sum_{j \in N_i} |P_j|}$.

Remark 5.1: (Handling feasibility in practice): Problem (18) is feasible if $z_{N_i} = z_{N_i}^{*,\epsilon}(\xi)$: $u_i^{*,\epsilon}(\xi)$ is a solution of (18) because of Proposition 2.1 and Assumption 2. Hence, if z_{N_i} is close to $z_{N_i}^{*,\epsilon}(\xi)$, then (18) is often feasible. Moreover, in practice, one can tune α_c^{ij} and α_k constraints in (14) only when they are close to being active to reduce the number of overall constraints and facilitate feasibility. •

Assumption 4: (Availability of optimizer in real time): The function $\bar{u}_i$ is instantaneously available to agent i for all $i \in [N]$.

Assumption 4 is a reasonable abstraction of what happens in practical scenarios, such as (14), which is a quadratic program

and can be solved efficiently [26]. In fact, if $M \leq 2$, $\bar{u}$ can even be found in closed form [27, Theorem 1].

Assumption 5: (Lipschitzness of optimizer): The functions $u^{*,\epsilon}$, $z^{*,\epsilon}$ and $\bar{u}$ are locally Lipschitz.

Remark 5.2: (Conditions that ensure Lipschitzness): The works [28], [29] study different conditions under which the solution of parametric optimization problems such as (18) or (17) is locally Lipschitz. •

We next show that the controller $\bar{u}$ is **safe** and **asymptotically** converges to $u^{*,\epsilon}$ when implemented in conjunction with the projected saddle-point dynamics as in (19). This, together with its distributed character, means that it meets all the desired properties. For to the problem described in Section III, this means that $\bar{u}$ achieves obstacle and inter-agent collision avoidance, and asymptotically converges to the closest controller to $u_{\text{nom}} = [u_{\text{nom},1}, \dots, u_{\text{nom},N}]$ that satisfies the safety constraints. In particular, if the agents are far from any of the obstacles in the environment and the CBF constraints in (14) are inactive, $\bar{u}$ converges to u_{nom} and steers the *leader* towards the desired waypoint and the *followers* towards their formation positions.

Proposition 5.3: (Convergence of algorithm): Suppose that Assumptions 2–5 hold. Then,

- i) the controller $\bar{u}$ is **safe**, i.e., if the initial conditions of (19) are such that $g_i^k(\xi_{N_i}(0), \bar{u}_i(\xi_i(0), z_{N_i}(0))) \leq 0$ for all $k \in [p]$, then the trajectories of (19) satisfy

$$\sum_{i \in V(\mathcal{G}_k)} g_i^k(\xi_{N_i}(t), \bar{u}_i(\xi_i(t), z_{N_i}(t))) \leq 0, \quad (20)$$

for all $k \in [p]$ and $t \geq 0$;

- ii) if the origin is asymptotically stable for the dynamical system $\dot{\xi} = F(\xi, u^{*,\epsilon}(\xi))$ with Lyapunov function $V: \mathbb{R}^n \rightarrow \mathbb{R}$, and Γ is a Lyapunov sublevel set of V contained in its region of attraction, then, for any initial condition of (19) $c_0 := (\gamma_0, z_0, \lambda_0, \xi_0) \in \mathbb{R}^{nN} \times \mathbb{R}^q \times \mathbb{R}^q \times \mathbb{R}^{nN}$, with $\xi_0 \in \Gamma$, $\delta > 0$ and $\epsilon > 0$, there exist $\tau_{c_0, \delta, \epsilon} > 0$ and r_{ξ_0} such that, if $\max\{\|\gamma_0 - u^{*,\epsilon}(\xi_0)\|, \|z_0 - z^{*,\epsilon}(\xi_0)\|, \|\lambda_0 - \lambda^{*,\epsilon}(\xi_0)\|\} < r_{\xi_0}$, and $\tau < \tau_{c_0, \delta, \epsilon}$, then the trajectories of (19) are such that, for all $t > 0$,

$$\|\gamma(t) - u^{*,\epsilon}(\xi(t))\| \leq \delta, \quad \|z(t) - z^{*,\epsilon}(\xi(t))\| \leq \delta,$$

$$\|\lambda(t) - \lambda^{*,\epsilon}(\xi(t))\| \leq \delta, \quad \|\bar{u}(\xi(t), z(t)) - u^{*,\epsilon}(\xi(t))\| \leq \delta,$$

and there exists $T_{c_0, \delta, \epsilon} > 0$ such that $\|\xi(t)\| \leq \delta$ for all $t > T_{c_0, \delta, \epsilon}$.

Proof: First we show (i). By definition of $\bar{u}$, we have

$$g_i^k(\xi_{N_i}(t), \bar{u}_i(\xi_i(t), z_{N_i}(t))) + \sum_{j \in N_i \cap V(\mathcal{G}_k)} (z_j^k(t) - z_i^k(t)) \leq 0, \quad (21)$$

for all $i \in [N]$, $k \in [p]$ and $t \geq 0$. Adding (21) for all $i \in V(\mathcal{G}_k)$, we obtain (20) for all $k \in [p]$ and $t \geq 0$. Next we show (ii). Since the dynamics in (19) are not differentiable due to the presence of the $[\cdot]_+$ operator, the standard version of Tikhonov's theorem for singular perturbations [30, Theorem 11.2] is not applicable. Instead we use [31], which gives a Tikhonov-type singular perturbation statement for differential inclusions. For non-smooth ODEs like (19) we need to check the following assumptions. First, the dynamics (19) are well-defined because Assumption 3 guarantees that $\bar{u}(\xi, z)$ is well-defined for all $\xi \in \mathbb{R}^{nN}$ and $z \in \mathbb{R}^q$. Second, the dynamics (19) are locally

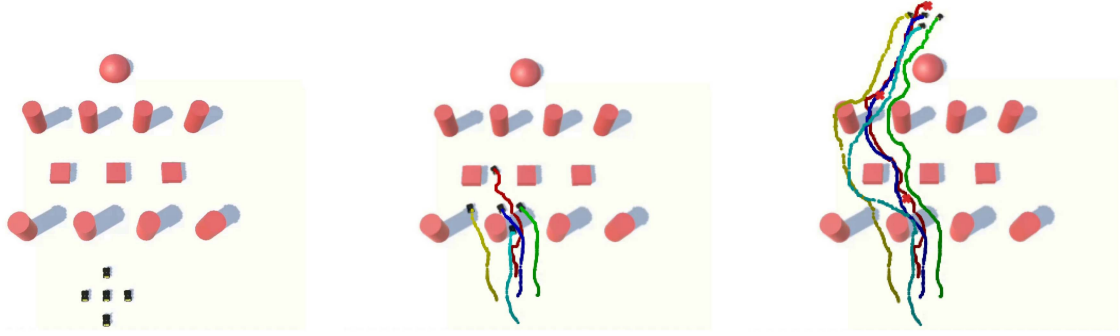


Fig. 2. Snapshots of the first simulation environment with color coded trajectories for the different robots. The intensity of the color decreases with time. In the last snapshot, the red x's indicate the three different waypoints for the leader of the team (in magenta). The environment has dimensions 20 m $\times$ 30 m.

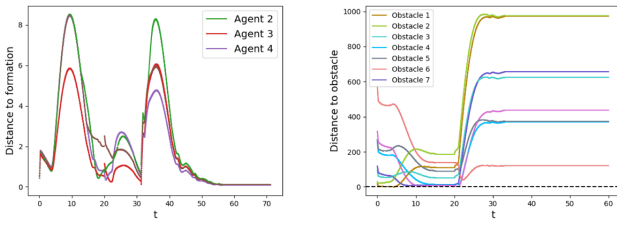


Fig. 3. Evolution in the first simulation environment, cf. Fig. 2. (left): Distance to the desired formation position over time for the different followers. (right): Distance to the different obstacles over time for the leader.

Lipschitz because of the Lipschitzness of the gradients of f and g , and the $\max$ operator, as well as the Lipschitzness of F_i for all $i \in \mathbb{R}^n$ and assumption 5. Third, the existence and uniqueness of the equilibrium of the fast dynamics follows from the fact that (23) has a strongly convex objective function and convex constraints, which implies that it has a unique KKT point. Fourth, Lipschitzness and asymptotic stability of the reduced-order model

$$\dot{\xi} = F(\xi, \bar{u}(\xi, z^{*,\epsilon}(\xi))). \quad (22)$$

Lipschitzness follows from Assumption 5, and asymptotic stability follows from the fact that $\bar{u}(\xi, z^{*,\epsilon}(\xi)) = u^{*,\epsilon}(\xi)$ for all $\xi \in \mathbb{R}^n$ (cf. Proposition 2.1) and the hypothesis that the origin of $\xi = F(\xi, u^{*,\epsilon}(\xi))$ is asymptotically stable. Fifth, the asymptotic stability of the fast dynamics for every fixed value of the slow variable follows from [19, Theorem 5.1]. Finally, the origin is the only equilibrium of (22) by assumption. The result follows from [31, Theorem 3.1 and Corollary 3.4], by adapting the results therein to the case where the origin of (22) has a bounded region of attraction. ■

If the constraints in (16) correspond to the CBF conditions of some set, as it is the case in (14), Proposition 5.3(i) implies that the set is forward invariant under the dynamics (19). Moreover, if $\xi = F(\xi, u^{*,\epsilon}(\xi))$ is globally asymptotically stable, then Proposition 5.3(ii) holds for any $\xi_0 \in \mathbb{R}^n$.

Remark 5.4: (Proximity of the constraint mismatch variables to their optimal values and choice of timescale): Proposition 5.3 requires that the initial conditions of γ , z and λ are close enough to $\gamma^{*,\epsilon}(\xi_0)$, $z^{*,\epsilon}(\xi_0)$ and $\lambda^{*,\epsilon}(\xi_0)$. This is due to the technical nature of the proof of [31, Theorem 3.1], which requires the fast variables to be in a small ball around the solution manifold to

show the stability of the interconnected system. The satisfaction of these conditions can be achieved by exploiting the asymptotic stability properties of the projected saddle-point dynamics and running them for the regularized version of (17) offline for a fixed value of the state ξ equal to ξ_0 within an accuracy smaller than r_{ξ_0} . Proposition 5.3 also requires τ to be sufficiently small. In practice we have observed that convergence of the state variables is achieved for a wide range of values of τ . Moreover, since safety holds for any τ , the value of τ can be decreased during the execution of the algorithm to ensure convergence to the desired waypoint. •

Remark 5.5: (Asymptotic stability assumption): Proposition 5.32 requires that the controller $u^{*,\epsilon}$ is asymptotically stabilizing. In the context of the problem outlined in Section III, this means that in a neighborhood of the origin, enforcing the obstacle avoidance and inter-agent collision avoidance constraints does not disrupt the stabilizing character of the nominal controllers (i.e., their steering towards the desired waypoints or desired formation positions of interest), which can be achieved by taking the values of α_c^{ij} and α_k sufficiently large. Recent work [23], [32] gives conditions under which the solution of a CBF-based QP of the form (14) with a nominal stabilizing controller retains its stability properties. Such conditions can be used to derive a subset contained in the region of attraction of the origin, in which the result in Proposition 5.3 2 can be applied. •

VI. EXPERIMENTAL VALIDATION

Here we show the performance of the proposed distributed control design (19) in simulation and in physical robotic platforms for a team of differential-drive robots, cf. Section III.

A. Parameter Tuning

Our experiments underscore the sensitivity of the control design to the choice of parameters. In particular, certain sequences of waypoints might lead to some of the agents of the team reaching deadlocks near the obstacles. This is a well-known issue of CBF-based controllers, cf. [23], [33]. In practice, we have observed that this behavior can be avoided by choosing a sequence of waypoints such that straight-lines connecting consecutive waypoints lie in the safe region, and promoting larger values of the angular velocity input, which allow the vehicle more *manoeuvrability*, by selecting the matrix Γ_i as the

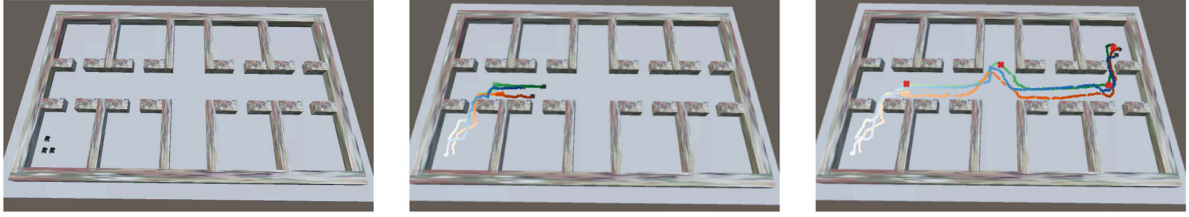


Fig. 4. Snapshots of the second simulation environment with color coded trajectories for the different robots. The intensity of the color increases with time. In the last snapshot, the red x's indicate the four different waypoints for the leader of the team. The environment has dimensions 20 m $\times$ 50 m.

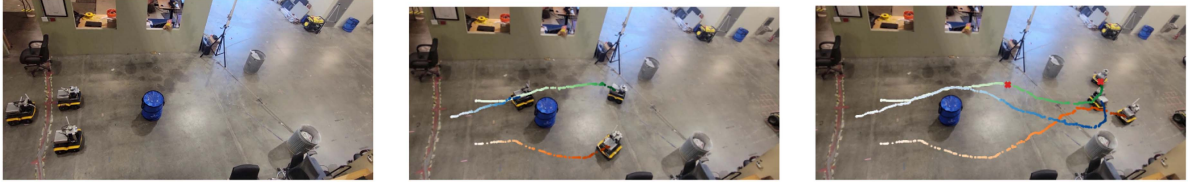


Fig. 5. Snapshots of the hardware experiment with color coded trajectories for the different robots. The intensity of the color increases with time. In the last snapshot, the red x's indicate the two different waypoints for the leader of the team. The environment has dimensions 4 m $\times$ 9 m.

constant matrix $[5, 0; 0, 1]$). We keep the other design parameters constant across the different experiments, with values $l = 0.2$, $\alpha_c^{ij} = 2.0$ for all $i \in [N]$, $j \in N_i$, $\alpha_k = 2.0$ for all $k \in [M]$, $d_{\min} = 1.0$, $\eta_k = 1.5$ for all $k \in [M]$, $\epsilon = 0.001$, and $\tau = 0.1$. Given the initial condition $x(0)$, we follow the procedure in Remark 5.4 to initialize the variables γ, z and λ in (19) before executing the controller. For each robot i , the set N_i is taken as the two closest robots to agent i in the initial positions.

B. Experiments

We have tested our algorithm in different simulation and hardware environments. For the simulation environments we have employed a high-fidelity Unity simulator on an Ubuntu Laptop with Intel Core i7-1355 U (4.5 GHz). We numerically integrate (19) and implement $\bar{u}$ using the convex optimization library CVXOPT [34]. The first simulation environment consists of a series of red cylindrical, cubic, and spherical obstacles. A team of 5 Husky¹ robots initially in a x-like formation traverses the environment while avoiding collision with obstacles and other robots of the team.² The simulated robots have the same LIDAR and sensor capabilities as the real ones, and these are used to run a SLAM system that allows each robot to localize itself in the environment and obtain its current state, which is needed to run (19) and implement $\bar{u}$. Fig. 2 shows 3 snapshots of the experiment and the trajectories followed by the robots. The robots successfully complete the task by avoiding collisions and reaching all the waypoints while maintaining the desired formation. Fig. 3 shows the evolution of the distance to the desired formation position for each follower and the distance to the different obstacles for the leader agent.

In the second simulation, initially a team of three Husky robots are located in one of the rooms in the environment, cf. Fig. 4(left). The team traverses the environment while maintaining a triangular formation. The walls are modelled as obstacles using a set of ellipsoidal barrier functions. The team successfully completes the task by avoiding collisions while maintaining the desired formation.

We have also validated our design in hardware in a team of 3 Jackal¹ robots with GPS, IMU, and LIDAR sensors, which they use to run a SLAM system to localize itself in the environment. The team is initially positioned as shown in Fig. 5(left). The blue and grey cylinders are modelled as obstacles using CBFs. The team traverses the environment while maintaining a triangular formation. All computations are done onboard with the computers of each of the robots.

VII. CONCLUSION

We have proposed a distributed controller design based on the solution of state-dependent network optimization problems with coupling constraints under general assumptions. When interconnected with the network dynamics, our controller is guaranteed to converge to the solution of the network optimization problem and satisfy the constraints at all times. We have leveraged this framework to provide a distributed solution that achieves safe navigation for a team of differential-drive robots in environments with obstacles, avoiding collisions and maintaining a desired formation, using CBFs. We have illustrated its performance both in simulation and hardware. Future work will investigate conditions that ensure the feasibility of the optimization problem encoding the control design, determine explicit bounds on the timescale separation required for exact convergence, design sequences of waypoints that ensure the network optimization problem remains feasible, investigate heuristics to tune the design parameters for improved performance, and explore the

¹Spec. sheets for the Husky and Jackal robots can be found at <https://clearpathrobotics.com>

²Video of the simulation: <https://tinyurl.com/distributedcbfs>

extension to settings with partial knowledge of the environment and the obstacles.

The regularized version of (17) takes the following form:

$$\begin{aligned} \min_{u \in \mathbb{R}^{mN}, z \in \mathbb{R}^q} \quad & \sum_{i=1}^N f_i(\xi_i, u_i) + \epsilon \sum_{k=1}^p \sum_{j \in V(\mathcal{G}_k)} (z_j^k)^2, \\ \text{s.t.} \quad & g_i^k(\xi_{\mathcal{N}_i}, u_i) + \sum_{j \in \mathcal{N}_i \cap V(\mathcal{G}_k)} (z_i^k - z_j^k) \leq 0, \quad i \in V(\mathcal{G}_k), \quad k \in [p]. \end{aligned} \quad (23)$$

The next sensitivity result shows that the solution to (23) and (17) can be made close by choosing ϵ sufficiently small.

Lemma 1.1: (Sensitivity of regularized problem): Let u^* be continuous, and assume $u^{*,\epsilon}$ is continuous for all $\epsilon > 0$. Let $\mathcal{K} \subset \mathbb{R}^{nN}$ be compact and $\delta > 0$. Then, there exists $\bar{\epsilon}_{\mathcal{K},\delta} > 0$ such that if $\epsilon < \bar{\epsilon}_{\mathcal{K},\delta}$, then $\|u^{*,\epsilon}(\xi) - u^*(\xi)\| \leq \delta$ for all $\xi \in \mathcal{K}$.

Proof: By [17, Lemma 4.2], for each $\xi \in \mathcal{K}$, there exists $\bar{\epsilon}_{\xi,\delta} > 0$ such that if $\epsilon < \bar{\epsilon}_{\xi,\delta}$, then $\|u^{*,\epsilon}(\xi) - u^*(\xi)\| \leq \frac{\delta}{2}$. Now, since $u^{*,\epsilon}$ and u^* are continuous, there exists a neighborhood $\mathcal{N}_\xi$ of ξ such that $\|u^{*,\epsilon}(\hat{\xi}) - u^*(\hat{\xi})\| \leq \delta$ for all $\hat{\xi} \in \mathcal{N}_\xi$. Since $\cup_{\xi \in \mathcal{K}} \mathcal{N}_\xi$ is an open covering of the compact set $\mathcal{K}$, there exists a finite subcover, i.e., there exists $N_{\mathcal{K}} \in \mathbb{Z}_{>0}$ and $\{\xi_i\}_{i=1}^{N_{\mathcal{K}}}$ such that $\mathcal{K} \subset \cup_{i=1}^{N_{\mathcal{K}}} \mathcal{N}_{\xi_i}$. The result follows by letting $\bar{\epsilon}_{\mathcal{K},\delta} := \min_{i \in [N_{\mathcal{K}}]} \{\bar{\epsilon}_{\xi_i,\delta}\}$. ■

ACKNOWLEDGMENT

Pol Mestres did a research internship at the U.S. Army Combat Capabilities Development Command Army Research Laboratory in Adelphi, MD during the summer of 2023.

REFERENCES

- [1] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. IEEE Eur. Control Conf.*, 2019, pp. 3420–3431.
- [2] P. Wieland and F. Allgöwer, "Constructive safety using control barrier functions," *IFAC Proc. Volumes*, vol. 40, no. 12, pp. 462–467, 2007.
- [3] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.
- [4] R. W. Brockett, "Dynamical systems that sort lists, diagonalize matrices, and solve linear programming problems," *Linear Algebra Appl.*, vol. 146, pp. 79–91, 1991.
- [5] U. Helmke and J. B. Moore, *Optimization and Dynamical Systems*. Berlin, Germany: Springer, 1994.
- [6] A. Hauswirth, S. Bolognani, and F. Dörfler, "Projected dynamical systems on irregular, non-Euclidean domains for nonlinear optimization," *SIAM J. Control Optim.*, vol. 59, no. 1, pp. 635–668, 2021.
- [7] M. Colombino, E. Dall'Anese, and A. Bernstein, "Online optimization as a feedback controller: Stability and tracking," *IEEE Trans. Control Netw. Syst.*, vol. 7, no. 1, pp. 422–432, Mar. 2020.
- [8] M. Colombino, J. W. Simpson-Porco, and A. Bernstein, "Towards robustness guarantees for feedback-based optimization," in *Proc. IEEE Conf. Decis. Control*, 2019, pp. 6207–6214.
- [9] A. Allibhoy and J. Cortés, "Control barrier function-based design of gradient flows for constrained nonlinear programming," *IEEE Trans. Autom. Control*, vol. 69, no. 6, pp. 3499–3514, Jun. 2024.
- [10] A. Colot, Y. Chen, B. Cornélusse, J. Cortés, and E. Dall'Anese, "Optimal power flow pursuit via feedback-based safe gradient flow," *IEEE Trans. Smart Grid*, 2024, submitted. Available at <https://arxiv.org/abs/2312.12267>
- [11] U. Borrmann, L. Wang, A. D. Ames, and M. Egerstedt, "Control barrier certificates for safe swarm behavior," *IFAC-LettersOnLine*, vol. 48, no. 27, pp. 68–73, 2015.
- [12] L. Wang, A. Ames, and M. Egerstedt, "Safety barrier certificates for collisions-free multirobot systems," *IEEE Trans. Robot.*, vol. 33, no. 3, pp. 661–674, Jun. 2017.
- [13] X. Tan and D. V. Dimarogonas, "Distributed implementation of control barrier functions for multi-agent systems," *IEEE Contr. Syst. Lett.*, vol. 6, pp. 1879–1884, 2022.
- [14] V. N. Fernandez-Ayala, X. Tan, and D. V. Dimarogonas, "Distributed barrier function-enabled human-in-the-loop control for multi-robot systems," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 7706–7712.
- [15] X. Tan, C. Liu, K. H. Johansson, and D. V. Dimarogonas, "A continuous-time violation-free multi-agent optimization algorithm and its applications to safe distributed control," 2024, *arXiv:2404.07571*.
- [16] C. Liu, X. Tan, X. Wu, D. V. Dimarogonas, and K. H. Johansson, "Achieving violation-free distributed optimization under coupling constraints," 2024, *arXiv:2404.07609*.
- [17] P. Mestres and J. Cortés, "Distributed and anytime algorithm for network optimization problems with separable structure," in *Proc. IEEE Conf. Decis. Control*, 2023, pp. 5463–5468.
- [18] N. de Carli, P. Salaris, and P. R. Giordano, "Distributed control barrier functions for global connectivity maintenance," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 1–7.
- [19] A. Cherukuri, E. Mallada, S. Low, and J. Cortés, "The role of convexity in saddle-point dynamics: Lyapunov function and robustness," *IEEE Trans. Autom. Control*, vol. 63, no. 8, pp. 2449–2464, Aug. 2018.
- [20] A. Cherukuri and J. Cortés, "Distributed algorithms for convex network optimization under non-sparse equality constraints," in *Proc. Allerton Conf. Commun., Control Comput.*, 2016, pp. 452–459.
- [21] P. Glotfelter, I. Buckley, and M. Egerstedt, "Hybrid nonsmooth barrier functions with applications to provably safe and composable collision avoidance for robotic systems," *IEEE Robot. Automat. Lett.*, vol. 4, no. 2, pp. 1303–1310, Apr. 2019.
- [22] M. Aicardi, G. Casalino, A. Bicchi, and A. Balestrino, "Closed loop steering of unicycle-like vehicles via Lyapunov techniques," *IEEE Robot. Automat. Mag.*, vol. 2, no. 1, pp. 27–35, Mar. 1995.
- [23] P. Mestres and J. Cortés, "Optimization-based safe stabilizing feedback with guaranteed region of attraction," *IEEE Contr. Syst. Lett.*, vol. 7, pp. 367–372, 2023.
- [24] X. Xu, "Constrained control of input-output linearizable systems using control sharing barrier functions," *Automatica*, vol. 87, pp. 195–201, 2018.
- [25] X. X. Tan and D. V. Dimarogonas, "Compatibility checking of multiple control barrier functions for input constrained systems," in *Proc. IEEE Conf. Decis. Control*, 2022, pp. 939–944.
- [26] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Math. Program. Computation*, vol. 12, pp. 637–672, 2020.
- [27] X. Tan and D. V. Dimarogonas, "On the undesired equilibria induced by control barrier function based quadratic programs," *Automatica*, vol. 159, 2013, Art. no. 111359.
- [28] B. J. Morris, M. J. Powell, and A. D. Ames, "Continuity and smoothness properties of nonlinear optimization-based feedback controllers," in *Proc. IEEE Conf. Decis. Control*, 2015, pp. 151–158.
- [29] P. Mestres, A. Allibhoy, and J. Cortés, "Regularity properties of optimization-based controllers," 2023, *arXiv:2311.13167*.
- [30] H. Khalil, *Nonlinear Systems*, 3rd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 2002.
- [31] F. Watbled, "On singular perturbations for differential inclusions on the infinite interval," *J. Math. Anal. Appl.*, vol. 310, no. 2, pp. 362–378, 2005.
- [32] W. S. Cortez and D. V. Dimarogonas, "On compatibility and region of attraction for safe, stabilizing control laws," *IEEE Trans. Autom. Control*, vol. 67, no. 9, pp. 4924–4931, Sep. 2022.
- [33] M. F. Reis, A. P. Aguilár, and P. Tabuada, "Control barrier function-based quadratic programs introduce undesirable asymptotically stable equilibria," *IEEE Control Syst. Lett.*, vol. 5, no. 2, pp. 731–736, Apr. 2021.
- [34] M. S. Andersen, J. Dahl, and L. Vandenbergh, "CVXOPT: A python package for convex optimization, version 1.1.6," 2013. [Online]. Available cvxopt.org